



Real-Time Vehicle Detection & Traffic-Congestion Monitoring on the Raspberry Pi 5

A YOLOv8n detector compressed for the CPU-only edge with structured pruning and INT8 / FP16 quantization, for a Safe City adaptive traffic-signal application.

Field	Detail
Student	Engr Danial Azam
Roll No	PHDEE25003
Instructor	Prof Rehan Hafiz
Course	Embedded AI
Target Device	Raspberry Pi 5
Base Model	YOLOv8n (Ultralytics)

Contents

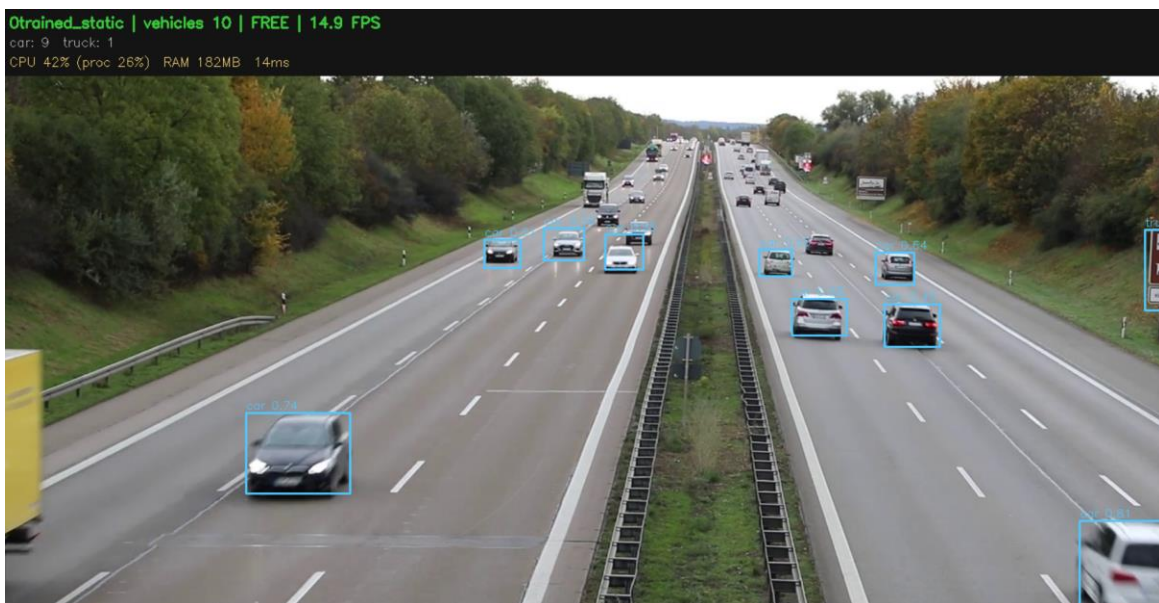
Contents	2
1 Abstract.....	3
2 Problem Statement	3
3 Dataset Details.....	4
4 Dataset Pre-processing.....	4
5 Baseline Model Architecture & Performance	4
6 Baseline Model Training / Inference Details	5
7 Hardware Details of Baseline Model Inference	7
8 Optimization Techniques Used.....	7
9 Comparative Results.....	9
10 Target Device Details	15
11 Deployment Challenges	15
12 Conclusion	16
13 Repository Link	16
14 References.....	17

(In Word: right-click the list → Update Field → Update entire table to fill page numbers.)

1 Abstract

This project builds an end-to-end embedded-AI system that detects vehicles and estimates road-traffic congestion in real time on a Raspberry Pi 5, intended for a Safe City deployment in which junction cameras drive adaptive traffic-signal timing.

A COCO-pretrained YOLOv8n detector is taken as the baseline (fine-tuned 20 epochs) and compressed for the CPU-only edge using two techniques: incremental structured channel pruning (10%, 20%, 30%) and INT8 / FP16 quantization. Sixteen ONNX variants are benchmarked on both a development laptop and the Pi 5 itself, measuring accuracy, latency, model size and resource usage at maximum speed. The best deployable result is the INT8 model with an FP16 head (04_trained_int8_fp16head) — full accuracy (~8.3 vehicles/frame), only 4.45 MB, and 19.9 ms / 50 FPS on the Pi (1.85x faster than FP32). A key finding is that INT8 is the only precision that actually accelerates on the Pi's Arm CPU; FP16 gives no CPU speed-up because ONNX Runtime casts it back to FP32.



Live detection on the Raspberry Pi 5 — INT8 model, vehicles detected, congestion FREE, real-time, with the on-frame CPU / RAM / latency overlay.

2 Problem Statement

Urban intersections in a Safe City programme are already fitted with cameras, but fixed-timing traffic signals cannot respond to changing load. The goal is to measure congestion per junction in real time — vehicle counts and a FREE / MODERATE / HEAVY state — and feed it to the signal controllers so green / red timings adapt automatically.

Streaming every camera's video to a central server is bandwidth-heavy and does not scale across a city. The alternative is inference at the edge: run the detector on a cheap device next to each camera. A Raspberry Pi 5 is the target, but a full-precision YOLOv8n is too heavy for its CPU to run at a useful frame rate. The core problem is therefore to compress the detector so it runs in real time on the Pi while preserving detection accuracy, and to quantify the size / speed / accuracy trade-offs of each optimization.

3 Dataset Details

Two data sources are used — one for training / calibration and one for in-context inference verification.

Dataset	Role	Notes
vehicles_coco	Baseline training, pruning recovery, INT8 calibration	548 images (469 train / 79 val) in COCO format, keeping COCO's class scheme so vehicles stay at car=2, motorcycle=3, bus=5, truck=7 — a larger, traffic-focused sample than the 128-image COCO128 default.
Demo highway clips	Accuracy / latency verification	Oblique road-camera traffic videos (highway-busy.mp4) matching the deployment viewpoint; accuracy = average vehicles/frame on the busy clip.

Dataset lesson — match viewpoint & class scheme A raw aerial drone set (VisDrone) transferred poorly to the oblique road-camera view, so the training sample (vehicles_coco) keeps an oblique road perspective and COCO's class indices — a 0-indexed custom set would make the COCO-pretrained model detect nothing until the IDs were remapped.

4 Dataset Pre-processing

Every frame passes through the same deterministic pipeline before inference:

- **Letterbox resize** to 320×320 — aspect ratio preserved, remaining area grey-padded (no distortion of vehicle shapes).
- **Colour + layout** — BGR→RGB, pixel values normalised to [0,1], reordered to NCHW.
- **Class filtering** — detections kept only for COCO ids 2 / 3 / 5 / 7 (vehicles).
- **INT8 calibration set** — 150 representative images from the vehicle dataset are run through the model so the static quantizer learns per-tensor activation ranges.
- **Pruning recovery data** — the same training set is reused for the post-pruning fine-tune.

5 Baseline Model Architecture & Performance

YOLOv8n (Ultralytics) is the nano member of the YOLOv8 family and the base model of this project: a single-stage, anchor-free object detector that predicts every box in one forward pass (no separate region-proposal stage) — the property that makes it fast enough for the CPU edge.

How YOLOv8 works

On each frame the network runs as one pipeline of three parts:

- **Backbone (CSP-Darknet)**. A stack of convolutional C2f blocks (cross-stage partial bottlenecks) extracts features at shrinking spatial resolutions, producing a 3-level feature pyramid (strides 8 / 16 / 32). C2f splits channels, sends part through bottleneck convolutions and concatenates — cheap but expressive; an SPPF block widens the receptive field at the end.
- **Neck (PAN-FPN)**. A top-down then bottom-up path-aggregation network fuses the three pyramid levels, so both small and large vehicles are well represented before detection.

- **Head (decoupled, anchor-free).** The detection head (model.22) attaches to the three fused scales and, at every grid cell, predicts a per-class score plus four box-edge distances. YOLOv8 is anchor-free — it regresses the box directly through a Distribution-Focal-Loss representation instead of matching pre-set anchor boxes — and the class and box branches are decoupled into separate conv stacks.

End-to-end: a letter-boxed 320×320 frame → backbone features → neck fusion → head predictions at three strides (~2100 candidate boxes at 320 px) → confidence threshold and non-max suppression → final boxes, then filtered to the four vehicle classes. This single-pass, anchor-free design together with the small nano width/depth (~3.2 M parameters, 8.7 GFLOPs) is exactly why YOLOv8n is the natural base model for a CPU-only edge device.

Reproduced performance vs. reported

Metric	Reported (Ultralytics, COCO val, 640 px)	Reproduced (this project)
Parameters	3.2 M	3.2 M
COCO mAP@50-95	37.3	see note
Input size	640×640	320×320 (edge)
FP32 model size (ONNX)	~12 MB	12.72 MB
Accuracy on busy-highway clip	—	~8.3 vehicles / frame
CPU latency (per frame, FP32)	~80 ms (ONNX-CPU ref.)	18.1 ms laptop · 36.9 ms Pi 5

Why we report a vehicles/frame proxy The published 37.3 mAP is measured on full COCO at 640 px. Our deployment runs at 320 px on road video for which there are no ground-truth labels, so the in-context accuracy metric is the average number of vehicles detected per frame on a fixed busy-highway clip — the quantity the Safe City controller actually consumes, used consistently to compare every optimized variant.

6 Baseline Model Training / Inference Details

Training (how the baseline is produced)

Training is transfer learning from COCO, not from scratch — the nano backbone already knows generic visual features, so only a short fine-tune is needed to specialise it to the traffic viewpoint:

- **Initialisation** — start from Ultralytics' COCO-pretrained `yo1ov8n.pt` weights.
- **Fine-tune** — train **20 epochs** on the 548-image `vehicles_coco` set at `imgsz 320` (the deployment resolution) with the Ultralytics defaults: auto-selected SGD/AdamW, a cosine learning-rate schedule, and on-the-fly augmentation (mosaic, HSV jitter, flips, scale). The objective is the YOLOv8 box (CIoU) + distribution-focal + classification loss. The result is `θtrained`, the 0%-pruned baseline.
- **Export** — convert the trained `.pt` to **ONNX, opset 13** (required for the per-channel INT8 QDQ used later) at a static 320×320 input, giving a fixed, fast graph.
- **Optimized variants** — pruned models are produced by iteratively removing channels and re-training (§8.1); quantized models are produced from the exported ONNX. Each is exported the same way.

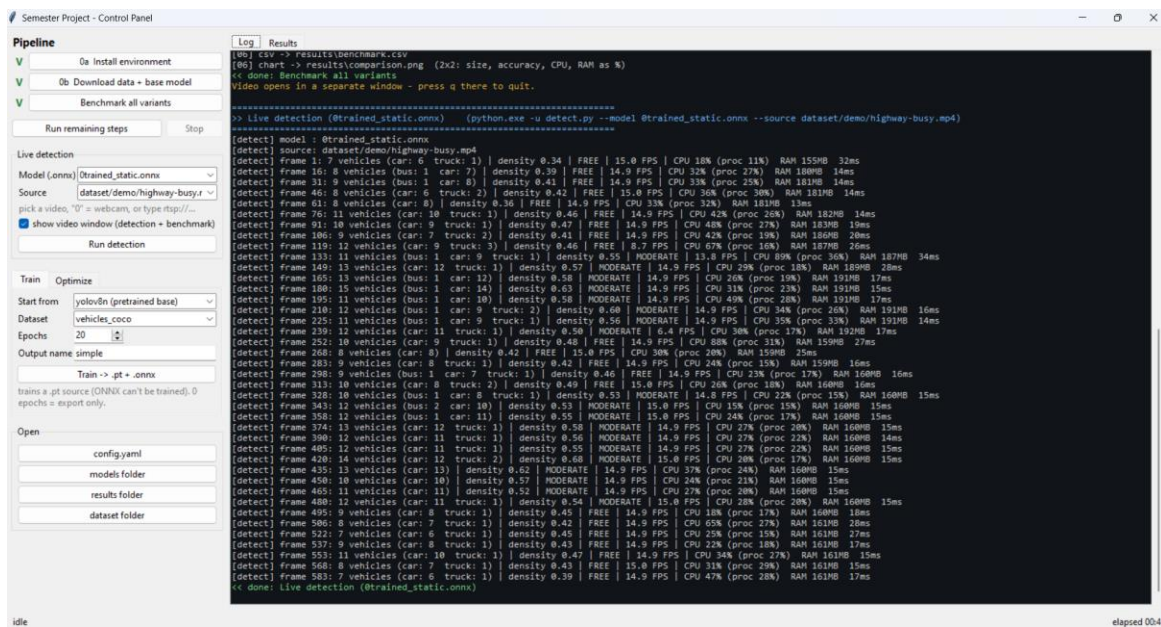
Accuracy measurement (mAP)

Accuracy is reported two ways: the in-context vehicles/frame proxy on the busy clip (used for every variant, including on the Pi), and proper COCO mAP@50-95 on the labelled validation split via the Ultralytics validator. mAP is measured on the PC only — it requires PyTorch + Ultralytics, which are deliberately not installed on the Raspberry Pi 5 (an inference-only, ~8 GB device), so the Pi benchmark leaves the mAP column blank and reports speed, size and vehicles/frame instead.

Inference

- **Runtime:** ONNX Runtime, CPU execution provider (4 threads; no GPU on the Pi).
- **Post-processing:** confidence threshold + non-max suppression, then vehicle-class filtering.
- **Congestion:** in-ROI vehicle density is smoothed and thresholded into FREE / MODERATE / HEAVY.
- **Instrumentation:** a live overlay reports vehicles, FPS and CPU / RAM / inference-ms; the benchmark runs each model uncapped at maximum speed and records its latency and peak FPS, so variants are compared by raw throughput.

The whole pipeline — train, prune, quantize, benchmark and live-detect — is driven from a single control-panel GUI (gui.py), with a headless launcher (run_on_pi.sh) for the Pi.



The Control Panel (gui.py): pipeline + live-detection picker + Train / Optimize tabs on the left; the live Log on the right.

7 Hardware Details of Baseline Model Inference

A PC does the heavy lifting (training, pruning, quantization, and the laptop benchmark); the Raspberry Pi 5 runs the deployed inference.

Role	Device	Details
Development / optimization	Laptop	Intel Core i7-1165G7 (Tiger Lake, 4 cores / 8 threads), CPU-only inference, Windows 11.
Edge deployment (target)	Raspberry Pi 5	Broadcom BCM2712 — quad-core Arm Cortex-A76 @ 2.4 GHz, LPDDR4X RAM, Raspberry Pi OS (64-bit, Bookworm), ONNX Runtime CPU. No GPU / NPU.

Both run inference purely on the CPU — the headline reason the study benchmarks on the Pi and not only on the laptop: INT8 acceleration behaves very differently on x86 vs. the Pi's Arm NEON cores.

8 Optimization Techniques Used

The exported FP32 ONNX is compressed for the CPU edge with two techniques, applied in order: structured pruning first (while the model is still trainable and can recover), then quantization (which freezes the graph).

8.1 Pruning — incremental, with recovery

Structured channel pruning (torch-pruning, grouped-L2 importance) removes the least-important convolution channels — and every weight that depends on them across the skip connections — so the network genuinely shrinks (unlike unstructured masking). The detection head's channel count is fixed by the class count and is left intact.

Rather than cut 10 / 20 / 30% in one shot, pruning is performed incrementally, which recovers far more accuracy:

- Prune **1%** of channels, then **fine-tune for 10 epochs** to recover.
- Repeat the prune-then-recover step: **10 rounds → 10% pruned; 20 rounds → 20%; 30 rounds → 30%** — so the 30% model is the result of 30 prune+fine-tune cycles.

Taking one small step at a time and letting the network heal after each step keeps it near a good optimum throughout, so the final 10 / 20 / 30% models hold accuracy that a single aggressive cut would lose — the incremental schedule measurably improved the recovered accuracy over one-shot pruning.

Critical finding — gentle recovery learning rate Each recovery fine-tune must use a low learning rate (0.0005). The default training rate (0.01) re-trains the network and erases the pretrained features — this single change took the pruned model from 0 back to a working detector.

8.2 Quantization — five levels tried, two that work

Quantization lowers the numeric precision of the weights (and, for INT8, the activations). Five levels were built from each exported model and tested on the CPU edge:

#	Quantization level	Result on the CPU edge
1	Dynamic INT8 (weights-only)	Runs, but little speed-up — activations stay float, so the heavy convolution math is not accelerated.
2	Pure / full INT8 (everything, incl. head)	Does not work — detections collapse to zero. Not used.
3	INT8 body + FP32 head (selective)	Works perfectly — full accuracy and real Arm speed-up. Deployable.
4	INT8 body + FP16 head	Works — same accuracy as #3 but the smallest file. Deployable.
5	FP16 (half precision, whole model)	No benefit on CPU — silently runs as FP32 (see below). Kept only as a size data-point.

Why pure INT8 breaks YOLO — and the fix The detection head (66 nodes under /model.22/) emits wide-range box / score values that 8-bit cannot represent; every prediction falls below threshold and detection drops to zero. Keeping the head in higher precision — FP32 (level 3) or FP16 (level 4), with the INT8 weight biases accumulated in INT32 — recovers the detections at essentially the same size. This INT8-body / float-head model is what we deploy.

Why FP16 gives nothing on the CPU — it silently becomes FP32 FP16 halves the file, but ONNX Runtime's CPU provider has no FP16 compute kernels: at load it converts the FP16 weights back to FP32 and inserts cast nodes, so the math runs in FP32 with no speed-up. We verified this directly — ORT's optimized graph shows the FP16 weights turned back to float and ~48 extra cast nodes — and on the Pi the FP16 model is actually slower than FP32 (52.7 ms vs 36.9 ms). True FP16 execution needs a GPU / NPU or a runtime with Arm FP16 kernels such as TFLite (XNNPACK) or ncnn; ONNX Runtime on the CPU cannot do it. INT8, by contrast, has real CPU kernels (Arm NEON dot-product), which is why it — not FP16 — is the speed win on the Pi.

Correct order: prune first (still trainable, can recover), then quantize (freezes the graph). So 30Pruned_static_fp16head is 30%-pruned then INT8-quantized with an FP16 head.

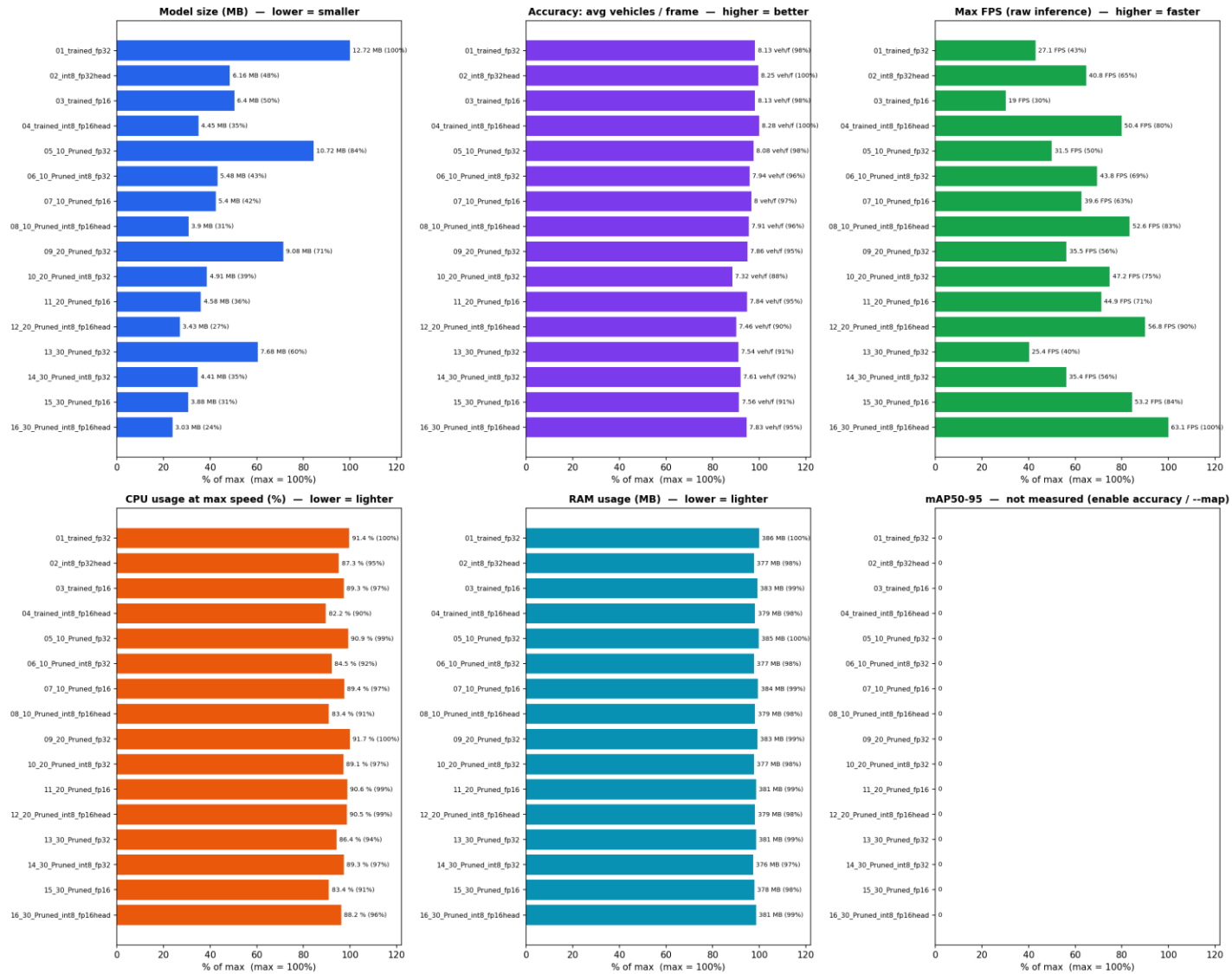
Hardware-aware NAS — investigated, not deployed A depth / width architecture search timed on ONNX Runtime was considered as a third technique, but its per-candidate retraining is GPU-intensive and impractical on the CPU-only workflow used here, so the deployed comparison covers the baseline plus pruning and quantization.

9 Comparative Results

All 16 ONNX variants — the baseline 0trained and the 10 / 20 / 30%-pruned models, each at _fp32, _fp16, INT8 (_static, FP32 head) and INT8+FP16-head (_static_fp16head) — were benchmarked uncapped at maximum speed on the busy-highway clip, first on the laptop, then on the Pi 5 itself. Each bar in the charts is a percentage of the maximum in its metric (max = 100%). mAP@50-95 is measured on the laptop only (PyTorch is not installed on the Pi).

9.1 On the Raspberry Pi 5 (real on-device numbers)

Model comparison on Raspberry Pi 5 — bars = % of max



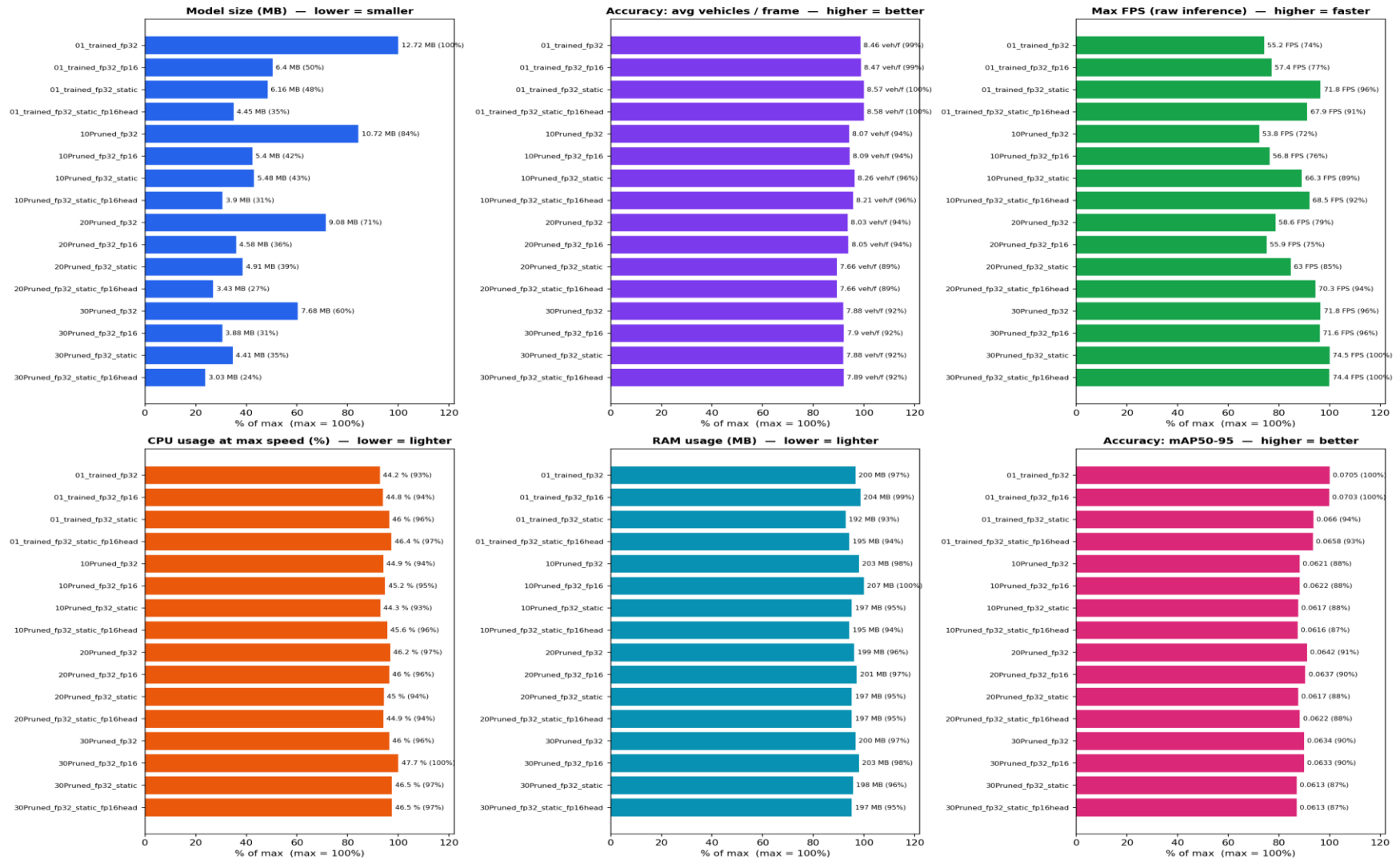
Pi 5 — size, accuracy (veh/frame), max FPS, CPU and RAM per variant (each bar as % of the max).

Model	Size (MB)	Pi infer (ms)	Max FPS	Acc (veh/f)	CPU %	RAM (MB)
01_trained_fp32.onnx	12.72	36.9	27.1	8.13	91.4	386.0
02_int8_fp32head.onnx	6.16	24.5	40.8	8.25	87.3	377.0
03_trained_fp16.onnx	6.4	52.7	19.0	8.13	89.3	383.0
04_trained_int8_fp16head.onnx	4.45	19.9	50.4	8.28	82.2	379.0
05_10_Pruned_fp32.onnx	10.72	31.8	31.5	8.08	90.9	385.0
06_10_Pruned_int8_fp32.onnx	5.48	22.8	43.8	7.94	84.5	377.0
07_10_Pruned_fp16.onnx	5.4	25.2	39.6	8.0	89.4	384.0
08_10_Pruned_int8_fp16head.onnx	3.9	19.0	52.6	7.91	83.4	379.0
09_20_Pruned_fp32.onnx	9.08	28.2	35.5	7.86	91.7	383.0
10_20_Pruned_int8_fp32.onnx	4.91	21.2	47.2	7.32	89.1	377.0
11_20_Pruned_fp16.onnx	4.58	22.3	44.9	7.84	90.6	381.0
12_20_Pruned_int8_fp16head.onnx	3.43	17.6	56.8	7.46	90.5	379.0
13_30_Pruned_fp32.onnx	7.68	39.3	25.4	7.54	86.4	381.0
14_30_Pruned_int8_fp32.onnx	4.41	28.3	35.4	7.61	89.3	376.0
15_30_Pruned_fp16.onnx	3.88	18.8	53.2	7.56	83.4	378.0
16_30_Pruned_int8_fp16head.onnx	3.03	15.8	63.1	7.83	88.2	381.0

Verdict on the Pi — INT8 wins, FP16 loses INT8 is the real speed-up on the Pi's Arm CPU: the selective INT8 model runs at 24.5 ms (40.8 FPS) versus 36.9 ms (27.1 FPS) for FP32 — ~1.5x faster — because Arm NEON executes 8-bit dot-products natively. Adding an FP16 head shrinks it to 4.45 MB at 19.9 ms (50.4 FPS), the fastest un-pruned model and the lowest CPU (82%). FP16, by contrast, is slower than FP32 on the Pi (52.7 ms vs 36.9 ms) — exactly the FP16-to-FP32 cast penalty from §8.2. Pruning on top takes the smallest model (30Pruned_int8_fp16head, 3.03 MB) to 15.8 ms (63 FPS).

9.2 On the development laptop (with mAP)

Model comparison on Laptop CPU — bars = % of max



Laptop — size, veh/frame, max FPS, CPU, RAM and mAP@50-95 per variant. INT8 still helps on the laptop, but its relative speed-up is smaller than on the Pi.

Model	Size (MB)	Latency (ms)	Max FPS	Acc (veh/f)	mAP50-95	CPU %	RAM (MB)
01_trained_fp32.onnx	12.72	18.1	55.2	8.46	0.0705	44.2	200.0
01_trained_fp32_fp16.onnx	6.4	17.4	57.4	8.47	0.0703	44.8	204.0
01_trained_fp32_static.onnx	6.16	13.9	71.8	8.57	0.066	46.0	192.0
01_trained_fp32_static_fp16head.onnx	4.45	14.7	67.9	8.58	0.0658	46.4	195.0
10Pruned_fp32.onnx	10.72	18.6	53.8	8.07	0.0621	44.9	203.0
10Pruned_fp32_fp16.onnx	5.4	17.6	56.8	8.09	0.0622	45.2	207.0
10Pruned_fp32_static.onnx	5.48	15.1	66.3	8.26	0.0617	44.3	197.0
10Pruned_fp32_static_fp16head.onnx	3.9	14.6	68.5	8.21	0.0616	45.6	195.0
20Pruned_fp32.onnx	9.08	17.1	58.6	8.03	0.0642	46.2	199.0
20Pruned_fp32_fp16.onnx	4.58	17.9	55.9	8.05	0.0637	46.0	201.0
20Pruned_fp32_static.onnx	4.91	15.9	63.0	7.66	0.0617	45.0	197.0
20Pruned_fp32_static_fp16head.onnx	3.43	14.2	70.3	7.66	0.0622	44.9	197.0
30Pruned_fp32.onnx	7.68	13.9	71.8	7.88	0.0634	46.0	200.0
30Pruned_fp32_fp16.onnx	3.88	14.0	71.6	7.9	0.0633	47.7	203.0
30Pruned_fp32_static.onnx	4.41	13.4	74.5	7.88	0.0613	46.5	198.0
30Pruned_fp32_static_fp16head.onnx	3.03	13.4	74.4	7.89	0.0613	46.5	197.0

9.3 Findings — reading both machines together

Dimension	What the two result sets show
INT8 is the only real speed-up	On the Pi, INT8 cuts latency ~1.5x (36.9→24.5 ms) and INT8+FP16-head ~1.85x (36.9→19.9 ms). On the x86 laptop INT8 also helps (18.1→13.9 ms) but the relative speed-up is smaller, which is why benchmarking on the target Arm hardware is essential.
FP16 is a trap on the CPU	Half the file but no compute speed-up — on the Pi it is the slowest variant (52.7 ms) because ORT casts it back to FP32. Useful only as storage, or for a GPU/NPU or a TFLite runtime.
Accuracy holds	Quantization barely moves accuracy (laptop mAP@50-95 0.0705 FP32 → 0.066 INT8; ~8.5 veh/frame either way). Incremental pruning holds up too: 10 / 20 / 30% stay ~0.061-0.064 mAP and ~7.7-8.3 veh/frame.
Size compounds	FP32 12.72 MB → INT8 6.16 MB (2x) → INT8+FP16-head 4.45 MB → 30%-pruned INT8+FP16-head 3.03 MB (4.2x smaller).
Resource use (Pi)	The INT8 variants need less CPU (82-90% vs ~91% FP32; the recommended model just 82%) at similar RAM (~377 MB), leaving headroom for the rest of the Safe City node.

Which model to deploy on the Raspberry Pi 5 Recommended: 04_trained_int8_fp16head (INT8 body + FP16 head, un-pruned) — 4.45 MB, 19.9 ms (50 FPS), full accuracy (8.28 veh/frame) and the lowest CPU load (82%); the best balance of speed, size and accuracy. If minimum size / maximum FPS matters more (many cameras per node): 16_30_Pruned_int8_fp16head — 3.03 MB, 15.8 ms (63 FPS), at a modest accuracy cost (~7.8 veh/frame, mAP 0.061 vs 0.066). Do not deploy plain FP16 on the Pi — it is slower than FP32 and pointless on the CPU.

10 Target Device Details

The deployment target is the Raspberry Pi 5 — an ~\$60-80 single-board computer that is a realistic per-junction edge node for a Safe City rollout.

Component	Specification
SoC	Broadcom BCM2712
CPU	Quad-core Arm Cortex-A76 @ 2.4 GHz (64-bit)
Memory	LPDDR4X-4267 (4 GB / 8 GB)
OS	Raspberry Pi OS (64-bit, Debian Bookworm)
Inference runtime	ONNX Runtime, CPU execution provider
Input	Local demo video (no camera / LAN / ONVIF required); RTSP optional

Deployment is a folder copy: install the lightweight inference dependencies (`python3 install.py --edge`) and run `bash run_on_pi.sh`, which auto-detects whether a display is present and otherwise runs headless.

11 Deployment Challenges

Getting the optimized models to run cleanly on the Pi surfaced several real, non-obvious problems:

Challenge	Cause & resolution
No video window on the Pi	pip's Arm opencv-python is a headless build (no <code>cv2.imshow</code>). Fixed by installing Debian's <code>python3-opencv</code> and exposing it to the venv with <code>--system-site-packages</code> .
Benchmark chart never generated	matplotlib was missing from the edge install; added to the Pi dependency set.
Chart wouldn't open	The Pi's default image viewer (Eye of MATE) was broken; the GUI now renders the chart in its own in-app window.
Full INT8 detected nothing	The detection head must stay FP32 (selective INT8) — quantizing it zeroes detections.
FP16 export produced an invalid graph	onnxconverter-common mishandled YOLO Resize/Concat; switched to the ONNX-Runtime transformers float16 converter.
INT8 graph rejected (axis for DequantizeLinear)	Per-channel INT8 needs opset 13; the export opset was raised from 12 to 13.
Pruned model detected nothing	Recovery fine-tune at the default LR destroyed the pretrained features; a gentle LR (0.0005) fixed it.
No GPU on the edge workflow	Hardware-aware NAS (which retrains each candidate) was not feasible and was left as future work.

12 Conclusion

The project delivers a complete, working embedded-AI pipeline: a COCO-pretrained YOLOv8n detector compressed for the Raspberry Pi 5 through structured pruning and INT8 / FP16 quantization, benchmarked on real hardware, and wrapped in a single GUI that trains, optimizes, benchmarks and runs live detection with a resource overlay.

Across 16 variants measured on the Pi, the clear recommendation is the INT8 model with an FP16 head (04_trained_int8_fp16head): full accuracy (~8.3 veh/frame), just 4.45 MB, and 19.9 ms / 50 FPS at the lowest CPU load — comfortably real-time on a ~\$70 device. INT8 is what actually accelerates on the Pi's Arm cores; FP16 gives no CPU speed-up (ONNX Runtime casts it back to FP32) and is even slower than FP32, while pure INT8 breaks the detector unless the head is kept in float. Incremental pruning shrinks the model further (to 3.03 MB) for a modest accuracy cost, so it is reserved for cases where every megabyte or frame counts. The work also surfaced reusable lessons: the incremental prune-then-recover schedule and its gentle learning rate, the detection head that must stay float under INT8, the FP16-becomes-FP32 limitation of ONNX Runtime on the CPU, and the necessity of benchmarking on the target Arm hardware.

For the Safe City application, each junction can run vehicle detection and congestion estimation locally on a Pi and report just the per-lane counts and congestion state to the traffic-signal controllers — low bandwidth, scalable, real-time. Future work: hardware-aware NAS on a GPU, a larger real traffic-camera dataset, and live integration with a signal-controller test-bed.

Prepared by Engr Danial Azam (PHDEE25003) for Prof Rehan Hafiz · Embedded AI.

13 Repository Link

The full code — baseline reproduction, optimization, and deployment — is in the project GitHub repository.

Repository: <https://github.com/DanialAzam/EmbeddedAI>

Clone: `git clone https://github.com/DanialAzam/EmbeddedAI.git`

Purpose	Code
Reproduce the baseline	download.py, 01_simple_model.py
Optimization	02_pruned_model.py, 03_quantized_model.py, quantize_levels.py
Benchmark	06_benchmark.py
Deployment / inference	detect.py, run_on_pi.sh, install.py
Control-panel GUI	gui.py

14 References

1. G. Jocher, A. Chaurasia, J. Qiu. Ultralytics YOLOv8, 2023. github.com/ultralytics/ultralytics
2. G. Fang et al. "DepGraph: Towards Any Structural Pruning" (Torch-Pruning). CVPR, 2023.
3. B. Jacob et al. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." CVPR, 2018.
4. Microsoft. ONNX Runtime. onnxruntime.ai
5. T.-Y. Lin et al. "Microsoft COCO: Common Objects in Context." ECCV, 2014.
6. Open Neural Network Exchange (ONNX). onnx.ai
7. Raspberry Pi Ltd. Raspberry Pi 5 product documentation. raspberrypi.com/documentation